

UCC 152-2

Introduction to security in DevOps

Workshop 2: Static security analysis of
DigiBank's source code

Cloud Computing Professional License

University of the Mountains

By **Engineer Willy Damtchou**

July 2026

Part 1: General framework of workshop 2

1.1 General context of the workshop

As part of the UCC152-2 Introduction to Security in DevOps course, Workshop 2 is the first step explicitly dedicated to the security analysis of the DigiBank application. After designing and implementing a first functional, modular, testable, and industrializable version of the system in Workshop 1, it is now possible to examine this foundation not only from the perspective of its operation, but also from that of its robustness against vulnerabilities present in the source code and in certain associated configurations.

This transition is important because it embodies a fundamental principle of DevSecOps: software must not only be able to execute its functionalities correctly, but it must also be built in such a way as to limit the introduction of exploitable vulnerabilities from the outset. From this perspective, the purpose of Workshop 2 is not to rebuild DigiBank, but to use the previously produced version as material for analysis, diagnosis, and remediation.

This workshop is a direct continuation of the Shift Left principle studied in the course. If security is to be considered as early as possible, then the source code itself must become a primary object of observation. This is precisely the role of SAST, or static security analysis, which allows for the examination of code even before the application is executed in order to identify certain categories of vulnerabilities, bad practices, or fragile structures that could compromise system security.

1.2 Pedagogical positioning of the workshop

From a pedagogical standpoint, this workshop marks a shift in the learner's approach. In workshop 1, the main focus was on building a clean, organized, coherent, and testable project. In workshop 2, the goal is no longer simply to produce code, but to read code as a space of risk, understand its weaknesses, interpret the alerts raised by analysis tools, and then correct the problems in a reasoned manner.

This development is essential in a DevSecOps curriculum. A student might know how to develop a functional application without necessarily being able to identify when input validation is insufficient, when a business error is reporting too much information,



when an outdated component introduces a risk, or when a code structure enables certain undesirable behaviors. Workshop 2 aims precisely to develop this ability to critically read code, not simply as technical text, but as a potential source of vulnerabilities.

The educational role of SAST should therefore not be reduced to the mere use of a tool. The goal is not for the student to passively launch a scan and collect a report. The goal is for them to learn to understand the meaning of the alerts, to distinguish truly important vulnerabilities from secondary or debatable alerts, to connect the analysis results to the theoretical concepts of the course, and then to produce justified corrections consistent with the DigiBank architecture.

1.3 Purpose of workshop 2

The purpose of workshop 2 is to conduct a complete static security analysis of the initial version of DigiBank, and then to correct the code and application configuration vulnerabilities that fall within the scope of SAST covered in the course.

In other words, the expected solution at the end of this workshop is not a new, different application, but a hardened, better controlled and technically safer version of the application built in workshop 1. This corrected version must maintain its good functional operation while reducing weaknesses detectable by reading the code, examining dependencies and analyzing application structures.

It is therefore important to understand that Workshop 2 is neither a purely theoretical audit nor a simple, opportunistic patching activity. It is a methodical process, rooted in DevSecOps principles, where code is examined early enough to prevent vulnerabilities from propagating through pipelines, containers, execution environments, or future iterations of the project.

1.4 DevSecOps Workshop Logic

This workshop provides a very concrete illustration of the shift from late-stage security to built-in security. In a traditional approach, many security flaws are only detected during final audits, penetration tests, or after anomalies appear in testing or production environments. This makes remediation more costly, slower, and more disruptive, as the weaknesses are already embedded in the architecture, components, and business logic.



Conversely, the approach advocated here consists of analyzing the code as close as possible to its point of introduction. SAST enables precisely this early detection. By observing the source code, certain flows, constructs, missing validations, or sensitive configurations, the team can identify risk signals very early on and intervene before these problems become more difficult to correct.

In line with the course's logic, workshop 2 therefore embodies one of the first technical levers of the Shift Left. It shows that security does not begin at deployment, nor when an application is already exposed, but from the moment the classes, methods, controllers, services, configuration files and dependencies embedded in the project are read.

1.5 Why DigiBank is a good support for a SAST workshop

The DigiBank project provides a particularly relevant framework for a static security analysis workshop, as it combines several characteristics typical of modern application systems. It handles potentially sensitive data, exposes REST APIs, implements business rules, relies on a relational database, integrates validations, depends on external libraries, and is already preparing for a form of industrialization via Docker and GitHub Actions.

These characteristics provide concrete material for static analysis. Even in a first educational version, such a project may present incomplete validations, overly detailed error messages, aging dependencies, overly visible secrets or test identifiers, insufficiently defined business rules, or structural choices that complicate access control or traceability of operations.

DigiBank's value also lies in its role as a central theme throughout the course. The vulnerabilities identified in Workshop 2 are not isolated from the rest of the course. They directly prepare the ground for Workshop 3, which focuses on DAST, and then Workshop 4, which addresses dependencies, containers, and delivery artifacts. The same project thus becomes a coherent field of observation, where security is progressively explored from several complementary perspectives.

1.6 Definition of static analysis within the workshop

In this workshop, static analysis refers to the examination of DigiBank's source code, bytecode, or certain compilation artifacts without actually running the application, with the aim of identifying potential vulnerabilities or code structures deemed risky. SAST therefore seeks to identify what, in the way the software is written and organized, could constitute a security weakness, regardless of whether the application appears to function correctly.

This distinction is important because an application can be fully compilable, pass its business tests, and produce the expected results while still containing significant weaknesses. SAST therefore doesn't simply verify that the code "works." It seeks to determine whether this code unnecessarily introduces risks related to data validation, application flows, error handling, dependencies, the handling of sensitive information, or certain coding practices.

In workshop 2, this analysis will rely on both automated tools and a reasoned reading of the project. The student's work should never be limited to simply copying the results of a scan. They must understand the logic behind the identified vulnerabilities, place them within the context of DigiBank, and assess their likely impact on the application's security.

1.7 General objective of the workshop

The overall objective of workshop 2 is to detect, analyze, prioritize and correct security vulnerabilities identifiable by static analysis in the DigiBank project, based on the SAST concepts studied in the course and on a DevSecOps approach oriented towards early prevention.

This overall objective involves four complementary dimensions. First, the student must know how to execute or utilize static analysis mechanisms. Second, they must be able to correctly interpret the results obtained. They must also produce relevant corrections without degrading the architecture or breaking the application's functional behavior. Finally, they must be able to justify the remediations performed based on clear security reasoning.

The workshop therefore does not only aim to raise alerts. It aims to transform these alerts into understanding, and then this understanding into concrete improvements to the code and configuration of the project.

1.8 Specific Objectives

At the end of this workshop, the student should be able to:

- **A2.1:** Explain the role of SAST in a DevSecOps approach and link it to the Shift Left principle
- **A2.2:** Analyze DigiBank's source code from a security perspective
- **A2.3:** Identify the main categories of static vulnerabilities present in the project
- **A2.4:** Use tools such as SonarQube, OWASP Dependency-Check, and other static quality mechanisms to support your diagnosis
- **A2.5:** Interpret analysis results and distinguish critical alerts from false positives or secondary signals
- **A2.6:** Correct the vulnerabilities detected in the code and in certain application configurations
- **A2.7:** Justify the remediation measures implemented from a technical and safety perspective.
- **A2.8:** Deliver a more robust version of DigiBank, ready to be subsequently submitted to the dynamic tests of workshop 3.

1.9 Scope of the analysis in workshop 2

The scope of this workshop covers all vulnerabilities or weaknesses that can reasonably be identified without actively running the application. This includes the Java code of business modules, REST controllers, services, entities, DTOs, exceptions, validations, application configuration files, certain Maven dependency definitions, and build artifacts directly related to the project.

However, this workshop does not yet cover observing the application's actual behavior in operation, manipulating sessions through running HTTP requests, or analyzing dynamic responses from outside the application. These aspects fall under the DAST (Dynamic Application Support Team) and will be addressed in the following workshop.

This distinction is pedagogically important. It avoids confusing two complementary approaches: workshop 2 examines the software's internal structure, while workshop 3 will observe its external behavior. The DigiBank project will thus serve successively as a case



study for internal and then external analysis, reinforcing the overall coherence of the learning path.

1.10 Types of vulnerabilities targeted

Within the DigiBank framework, the vulnerabilities or weaknesses sought through static analysis may include:

- Insufficient validation of user input
- Code structures that can facilitate certain injections or unsafe uses
- Overly revealing error messages or exceptions
- Secrets or identifiers visible in the configuration
- Outdated or known risky dependencies
- Fragile coding practices from a security standpoint
- Insufficient access controls when they already appear in the code structure or in the endpoint logic.
- Architectural or configuration choices that complicate risk reduction in a later phase.

It's important to note that some of these vulnerabilities don't necessarily mean that an exploit has already been proven. The role of SAST is precisely to flag potential weaknesses or concerning code patterns before they are tested through execution or an actual attack.

1.11 Tools to use

The tools selected for the workshop cover development, static analysis, dependency checking, test quality, and pipeline automation. They should be presented as complementary instruments, each intervening at a specific level of the code security chain.

- IntelliJ IDEA: is used to open, read, modify and run the DigiBank project. It is the student's main working environment.
- JDK Java: allows you to compile and run the project. It forms the technical basis of the Java environment.
- Maven: manages dependencies, compilation, testing, and analysis plugins. It also serves as an entry point for running SAST checks.

- Spring Boot: provides the application framework for the analyzed project. It allows for an understanding of the actual organization of the source code under examination.
- SonarQube: analyzes code quality and flags security, maintainability, and robustness issues. It is the primary static analysis tool in the shop floor.
- OWASP Dependency-Check: checks project dependencies to identify libraries with known vulnerabilities.
- PITest: evaluates the robustness of tests by measuring their ability to detect artificial code modifications. It helps to judge the true quality of test coverage.
- JUnit 5: allows you to run or complete unit tests after correction. It is used to verify that the remediations have not broken the expected behavior.
- Git: allows you to track changes made to the code and keep a clear record of security fixes.
- GitHub: hosts the project repository and centralizes the analyzed code. It also serves as a platform for automation.
- GitHub Actions: automates compilation, testing, and analysis checks in the CI/CD pipeline. It shows how to integrate SAST into the development workflow.

1.12 Expected result at the end of part 1

At the end of this first part, the student should have understood why workshop 2 exists, what a static security analysis really is, how it extends the DevSecOps logic studied in the course, and why DigiBank is a relevant field for this practical application.

He must also be able to explain:

- Why does SAST intervene before the application is executed?
- How does it embody the principle of Shift Left?
- What categories of vulnerabilities can be identified in the source code?
- Why the human interpretation of the results is as important as the tool itself
- And how this workshop logically prepares the future steps of the journey, in particular the DAST applied to the corrected version of DigiBank.

The next part can then move into the operational work itself: preparation of the analysis environment, choice of tools, execution of the first static checks, observation of the results and organization of the remediation.



Part 2: Practical Preparation of DigiBank's SAST Environment

In this second part of Workshop 2, the goal is not yet to fix DigiBank's vulnerabilities, but to set up a reliable, reproducible, and consistent analysis environment. This step is crucial because a serious SAST workshop doesn't begin with blindly running tools, but with rigorous preparation of the project, Maven plugins, scanners, reports, and the pipeline that will then be used to automate the evaluation of code quality and security.

In keeping with the spirit of the course, this phase embodies the Shift Left principle: security checks should not be postponed until the end of the cycle, but integrated into the regular practices of building, testing, and reviewing code. SonarQube, OWASP Dependency-Check, PITest, and GitHub Actions are therefore not presented here as isolated tools, but as components of an initial static control chain applied to the DigiBank project.

2.1 Role of this phase

Before launching any analysis, the student must start with a functional version of DigiBank, as built in Lab 1, including its Maven architecture, modules, dependencies, unit tests, and Spring Boot entry point. The Lab 1 PDF clearly demonstrates that DigiBank was designed from the outset to prepare for subsequent workshops, notably through the early integration of tests, API documentation, and automation tools such as Docker and GitHub Actions.

This section therefore has a dual purpose. On the one hand, it restores the project to a clean state before analysis; on the other hand, it installs the building blocks of static analysis which will then be used to identify flaws in the code, dependencies and the robustness of the tests.

2.2 Software Prerequisites

The student must have a fully functional workstation for Java and Maven development, in accordance with the recommendations of Lab 1, which standardizes the use of Java 17, Maven 3.9.x, recent IntelliJ IDEA, Git, PostgreSQL, and Docker within the DigiBank environment. This version consistency is important to minimize potential problems related to tool incompatibilities.

For Workshop 2, the following components must be installed or checked:

- Java JDK 17.
- Maven 3.9.x or compatible.
- IntelliJ IDEA Community or Ultimate.
- Git.
- Docker Desktop or Docker Engine with Docker Compose.
- A modern browser.
- Possibly Postman, although this part remains focused on SAST.
- Access to GitHub to push the repository and run the GitHub Actions pipeline.
- SonarScanner is not mandatory if the analysis is launched via Maven, which will be the case here.

The basic verification commands are as follows:

```
Java -version
mvn -version
git --version
docker --version
docker compound version
```

If any of these commands fail, the problem must be corrected before proceeding. A static analysis workbench can quickly become misleading if the environment is not properly managed, as a build failure or a poor Maven resolution can be mistaken for a security alert.

2.3 Launch of the DigiBank project

The DigiBank project must be the one built during Workshop 1, that is, a multi-module Maven parent structured around a modular monolith. Lab 1 indicates that the project root is a Maven parent, digibank-parent, controlling several sub-modules such as common-module, customer-module, account-module, transfer-module, and digibank-web, all assembled into a single application.

In IntelliJ IDEA, the project must be opened properly:

1. Open IntelliJ IDEA.
2. Choose Open.
3. Select the root folder digibank-parent.
4. Verify that the IDE detects a Maven project.
5. Wait for full indexing.
6. Open the Maven view and click on Reload All Maven Projects.
7. Verify that all modules are visible and that Spring Boot dependencies are correctly resolved.

At this stage, the student must run a test build from the project root:

```
CD digibank-parent  
mvn clean install
```

This command should rebuild the project and run the existing tests. In Lab 1, this command is already presented as the standard check for the overall consistency of the multi-module project. If it fails, the structure or dependencies must be corrected before introducing the SAST tools.

2.4 Git backup before instrumentation

Before adding analysis plugins and security configurations, it is recommended to create a Git restore point. This practice is consistent with the DevSecOps logic of the course,

which emphasizes traceability, reproducibility, and the ability to revert to a stable project state.

Execute :

```
git status  
  
git add .  
  
git commit -m "Baseline before SAST tooling configuration"
```

This backup will allow us to clearly distinguish the initial state of DigiBank and the instrumented state for Workshop 2.

2.5 Selected configuration principle

In this section, the tools will be integrated primarily at the Maven level, allowing students to run them locally with simple commands and reuse them almost identically in GitHub Actions. This approach is pedagogically sound, as it demonstrates that the same project can be controlled simultaneously in the IDE, via the command line, and in a CI/CD pipeline without multiplying configuration mechanisms.

The chosen approach will be as follows:

- **SonarQube** for overall quality, technical debt, certain code vulnerabilities, and cross-cutting metrics.
- **OWASP Dependency-Check** for analyzing known vulnerable dependencies.
- **PITest** for mutation testing, useful here as an indicator of test robustness and ability to detect logical defects.
- **GitHub Actions** to automate the execution of builds, tests, dependency scans and, if desired, SonarQube analysis.

2.6 Update of the parent pom.xml

The cleanest approach is to centralize versions and plugins at the Maven parent level. Lab 1 already demonstrates that DigiBank's Maven parent is used to unify Java and Spring Boot versions, common dependencies, and multi-module building; therefore, it makes sense to add the versions of the control tools from Workshop 2 to it.

In the root pom.xml file, add or complete the properties section as follows:

```
<properties>

<java.version> 17 </java.version>

<spring.boot.version> 3.3.2 </spring.boot.version>

<springdoc.version> 2.6.0 </springdoc.version>

<cucumber.version> 7.18.0 </cucumber.version>

<junit.version> 5.10.2 </junit.version>

<maven.compiler.plugin.version> 3.13.0
</maven.compiler.plugin.version>

<sonar.maven.plugin.version> 3.11.0.3922
</sonar.maven.plugin.version>

<dependency-check.version> 10.0.4 </dependency-check.version>

<pitest.version> 1.16.2 </pitest.version>

<pitest.junit5.plugin.version> 1.2.1
</pitest.junit5.plugin.version>

<surefire.version> 3.2.5 </surefire.version>

</properties>
```

These features make the project more readable and prevent version numbers from being scattered across multiple modules. In an educational context, this centralization also facilitates future updates and the annotated review of the build in class.

2.7 SonarQube Maven Configuration

SonarQube can be launched via the Maven plugin sonar-maven-plugin, which saves the student from having to install a separate scanner. In a multi-module project like DigiBank, this approach is particularly advantageous, as the analysis is launched from the root directory and aggregates the results from the different modules.

Still within the parent pom.xml file, add the following configuration to build > pluginManagement > plugins or directly to build > plugins:

```
<pluginManagement>

<plugins>

<plugin>

<groupId> org.apache.maven.plugins </groupId>

<artifactId> maven-compiler-plugin </artifactId>

<version> ${maven.compiler.plugin.version} </version>

<configuration>

<source> ${java.version} </source>

<target> ${java.version} </target>

</configuration>

</plugin>

<plugin>

<groupId> org.sonarsource.scanner.maven </groupId>

<artifactId> sonar-maven-plugin </artifactId>

<version> ${sonar.maven.plugin.version} </version>

</plugin>

</plugins>

</pluginManagement>
```

You can also add basic SonarQube properties to the same pom.xml file:

```
< properties >
```

```
< sonar.projectKey > digibank-parent </ sonar.projectKey >

< sonar.projectName > DigiBank Parent </ sonar.projectName >

< sonar.host.url > http://localhost:9000 </ sonar.host.url >

</ properties >
```

If the student wants to avoid writing certain values in the POM, particularly for more realistic security reasons, it's preferable to pass the token and sometimes even the URL via environment variables or command-line parameters. The course also emphasizes that secrets or tokens in configuration files are precisely a vulnerability to analyze in DigiBank.

2.8 Installing and launching SonarQube with Docker

To maintain a lightweight and reproducible installation, the simplest approach is to run SonarQube with Docker. This approach aligns well with the recommendations of Lab 1, which already advocated using Docker to create reproducible environments, particularly for PostgreSQL.

Create a local working folder for SonarQube if needed, then launch:

```
docker network create digibank-net

docker volume create sonarqube_data

docker volume create sonarqube_logs

docker volume create sonarqube_extensions
```

Next, launch SonarQube:

```
docker run -d \

  --name sonarqube \

  --network digibank-net \
```

```
-p 9000:9000 \  
  
-v sonarqube_data:/opt/sonarqube/data \  
  
-v sonarqube_logs:/opt/sonarqube/logs \  
  
-v sonarqube_extensions:/opt/sonarqube/extensions \  
  
sonarqube:lts-community
```

Check the container status:

```
docker ps  
  
docker logs -f sonarqube
```

Once the service is ready, open **http://localhost:9000** . The default credentials are typically **admin/admin** , but SonarQube usually prompts you to change your password on first access. Set the new password to **digibank123** . After logging in, you need to create a local DigiBank project or use the key defined in Maven, and then generate a user token to use for analysis.

2.9 SonarQube Analysis Command

Once SonarQube is accessible and the token obtained, the analysis can be launched from the DigiBank root:

```
mvn clean verify sonar:sonar \  
  
-Dsonar.projectKey=digibank-parent \  
  
-Dsonar.projectName= "DigiBank Parent" \  
  
-Dsonar.host.url=http://localhost:9000 \  
  
-Dsonar.login=YOUR_TOKEN_SONAR
```

This command rebuilds the project, runs the Maven checks, and then pushes the metrics and results to the SonarQube server. It already allows the student to observe code smells, duplications, potential vulnerabilities, security hotspots, and coverage metrics if the tests are correctly integrated.

In an educational context, it's helpful to explain that SonarQube isn't an absolute oracle. The course specifically emphasizes that the value of a static analysis tool lies not in simply generating alerts, but in the ability to interpret and prioritize the results.

2.10 Configuring OWASP Dependency-Check

The DigiBank course and case study emphasize that some backend libraries are outdated or vulnerable, and that dependency analysis is an integral part of the DevSecOps approach. OWASP Dependency-Check is therefore a tool perfectly suited to the logic of Workshop 2.

In the parent pom.xml file, add the following plugin in build > plugins:

```
<plugin>

<groupId> org.owasp </groupId>

<artifactId> dependency-check-maven </artifactId>

<version> ${dependency-check.version} </version>

<configuration>

<format> HTML </format>

<failBuildOnCVSS> 7 </failBuildOnCVSS>

<skipProvidedScope> true </skipProvidedScope>

<skipTestScope> true </skipTestScope>

<nvdApiKey> ${env.NVD_API_KEY} </nvdApiKey>

<ossindexAnalyzerEnabled> false </ossindexAnalyzerEnabled>

<deletionFiles>
```

```
<suppressionFile>                                dependency-check-suppressions.xml
</suppressionFile>

</suppressionFiles>

</configuration>

<executions>
<execution>

<phase> verify </phase>

<goals>
<goal> check </goal>
</goals>
</execution>
</executions>

</plugin>

<plugin>

<groupId> org.owasp </groupId>

<artifactId> dependency-check-maven </artifactId>

</plugin>
```

This configuration instructs Maven to perform a dependency scan and generate an HTML report. The failBuildOnCVSS threshold of 7 allows the build to fail if a vulnerability deemed high or critical is detected, immediately introducing automated safeguards into the project.

The NVD API key, obtained via the NIST portal, accelerates OWASP Dependency-Check scans by allowing optimized access to the NVD CVE database and reducing download limitations.

Generation link:

```
https://nvd.nist.gov/developers/request-an-api-key
```

2.11 OWASP Dependency-Check Commands

To start the scan locally, run:

```
mvn org.owasp:dependency-check-maven:check
-DnvdApiKey=YOUR_NVD_API_KEY
```

Or, if the plugin is already well integrated into the cycle:

```
mvn clean verify
```

To purge the owasp database, run

```
mvn org.owasp:dependency-check-maven:purge
```

The report is usually produced in a file of the following type:

```
target/dependency-check-report.html
```

In a multi-module project, several reports may appear depending on how the modules are analyzed. It is advisable to check the parent module's report and, if necessary, those of the modules with the most application dependencies.

2.12 PITest Configuration

Lab 1 indicates that unit tests are present from the very first workshop, particularly to verify business rules such as the correct creation of a customer, the opening of an account, or the refusal of a transfer in case of insufficient funds. PITest then becomes very relevant in Workshop 2, because it doesn't just measure that tests exist: it evaluates their actual ability to detect logical changes in the code.

In the parent pom.xml file, add the following plugin:

```
<plugin>

<groupId> org.pitest </groupId>

<artifactId> pitest-maven </artifactId>

<version> ${pitest.version} </version>

<dependencies>

<dependency>

<groupId> org.pitest </groupId>

<artifactId> pitest-junit5-plugin </artifactId>

<version> ${pitest.junit5.plugin.version} </version>

</dependency>

</dependencies>

<configuration>

<targetClasses>

<param> com.m2ibank.*.* </param>

</targetClasses>

<targetTests>

<param> com.m2ibank.*.* </param>

</targetTests>

<threads> 4 </threads>

<outputFormats>

<param> HTML </param>

<param> XML </param>

</outputFormats>

<timestampedReports> false </timestampedReports>


```

```
</configuration>

</plugin>
```

In some cases, on a multi-module system, it can be useful to refine `targetClasses` to explicitly target business packages. For example, you can target `com.m2ibank.customer.*`, `com.m2ibank.account.*`, and `com.m2ibank.transfer.*` to focus the analysis on business services and components.

2.13 PITest Command

Launch PITest with:

```
mvn org.pitest:pitest-maven:mutationCoverage
```

The HTML report is then generated in a directory of the following type:

```
target/pit-reports
```

The pedagogical value here is significant: a student might have seemingly adequate security coverage but a low mutation score, indicating that the tests are being passed without truly protecting the business rules. This directly aligns with the spirit of Lab 1, which aimed for DigiBank to be not only executable but also sufficiently clean and testable for subsequent security workshops.

2.14 Surefire Recommended Configuration

To stabilize the execution of JUnit 5 tests and avoid certain ambiguities in analysis tools, it is useful to explicitly add the Surefire plugin to the Maven parent:

```
<plugin>

<groupId> org.apache.maven.plugins </groupId>
```

```
<artifactId> maven-surefire-plugin </artifactId>

<version> ${surefire.version} </version>

<configuration>

<useModulePath> false </useModulePath>

</configuration>

</plugin>
```

This configuration limits certain execution difficulties depending on the Java environment and ensures more stable test behavior in Maven as well as in GitHub Actions.

2.15 Consolidated example of the parent pom.xml

Here is a consolidated version of the central part of the parent pom.xml file, adapted for Workshop 2:

```
<project xmlns = "http://maven.apache.org/POM/4.0.0"
    xmlns: xsi = "http://www.w3.org/2001/XMLSchema-instance"
    xsi :schemaLocation = "http://maven.apache.org/POM/4.0.0
https://maven.apache.org/xsd/maven-4.0.0.xsd

<modelVersion> 4.0.0 </modelVersion>

<groupId> com.m2ibank </groupId>

<artifactId> digibank-parent </artifactId>

<version> 1.0.0-SNAPSHOT </version>

<packaging> pom </packaging>

<name> digibank-parent </name>
```

```
<description>    DigiBank    modular    monolith    parent    project
</description>

<modules>

<module> common-module </module>

<module> customer-module </module>

<module> account-module </module>

<module> transfer-module </module>

<module> digibank-web </module>

</modules>

<properties>

<java.version> 17 </java.version>

<spring.boot.version> 3.4.3 </spring.boot.version>

<springdoc.version> 2.6.0 </springdoc.version>

<cucumber.version> 7.18.0 </cucumber.version>

<junit.version> 5.10.2 </junit.version>

<maven.compiler.plugin.version>                                3.13.0
</maven.compiler.plugin.version>

<sonar.maven.plugin.version>                                    3.11.0.3922
</sonar.maven.plugin.version>

<dependency-check.version> 10.0.4 </dependency-check.version>

<pitest.version> 1.16.2 </pitest.version>

<pitest.junit5.plugin.version>                                  1.2.1
</pitest.junit5.plugin.version>
```

```
<surefire.version> 3.2.5 </surefire.version>

<sonar.projectKey> digibank-parent </sonar.projectKey>

<sonar.projectName> DigiBank Parent </sonar.projectName>

<sonar.host.url> http://localhost:9000 </sonar.host.url>

</properties>


<dependencyManagement>

<dependencies>

<dependency>

<groupId> org.springframework.boot </groupId>

<artifactId> spring-boot-dependencies </artifactId>

<version> ${spring.boot.version} </version>

<type> pom </type>

<scope> import </scope>

</dependency>

</dependencies>

</dependencyManagement>


<build>

<pluginManagement>

<plugins>

<plugin>

<groupId> org.apache.maven.plugins </groupId>
```

```
<artifactId> maven-compiler-plugin </artifactId>

<version> ${maven.compiler.plugin.version} </version>

<configuration>

<source> ${java.version} </source>

<target> ${java.version} </target>

</configuration>

</plugin>

<plugin>

<groupId> org.sonarsource.scanner.maven </groupId>

<artifactId> sonar-maven-plugin </artifactId>

<version> ${sonar.maven.plugin.version} </version>

</plugin>

</plugins>

</pluginManagement>

<plugins>

<plugin>

<groupId> org.apache.maven.plugins </groupId>

<artifactId> maven-surefire-plugin </artifactId>

<version> ${surefire.version} </version>

<configuration>

<useModulePath> false </useModulePath>

</configuration>

</plugin>

<plugin>
```

```
<groupId> org.owasp </groupId>

<artifactId> dependency-check-maven </artifactId>

<version> ${dependency-check.version} </version>

<configuration>

<format> HTML </format>

<failBuildOnCVSS> 7 </failBuildOnCVSS>

<skipProvidedScope> true </skipProvidedScope>

<skipTestScope> true </skipTestScope>

<nvdApiKey> ${env.NVD_API_KEY} </nvdApiKey>

<ossindexAnalyzerEnabled> false </ossindexAnalyzerEnabled>

<deletionFiles>

<suppressionFile>                                dependency-check-suppressions.xml
</suppressionFile>

</suppressionFiles>

</configuration>

<executions>

<execution>

<phase> verify </phase>

<goals>

<goal> check </goal>

</goals>

</execution>

</executions>

</plugin>
```

```
<plugin>

<groupId> org.pitest </groupId>

<artifactId> pitest-maven </artifactId>

<version> ${pitest.version} </version>

<dependencies>

<dependency>

<groupId> org.pitest </groupId>

<artifactId> pitest-junit5-plugin </artifactId>

<version> ${pitest.junit5.plugin.version} </version>

</dependency>

</dependencies>

<configuration>

<targetClasses>

<param> com.m2ibank.*.* </param>

</targetClasses>

<targetTests>

<param> com.m2ibank.*.* </param>

</targetTests>

<threads> 4 </threads>

<outputFormats>

<param> HTML </param>

<param> XML </param>

</outputFormats>

<timestampedReports> false </timestampedReports>


```

```
</configuration>

<executions>

<execution>

<id> pitest </id>

<phase> verify </phase>

<goals>

<goal> mutationCoverage </goal>

</goals>

</execution>

</executions>

</plugin>

</plugins>

</build>

</project>
```

This configuration gives DigiBank's Maven parent a true SAST orchestrator role, which is consistent with the multi-module logic explained in TP1.

2.16 Maven commands for this phase

Once the configuration is added, the important commands in this section are as follows:

```
mvn clean

mvn compile

mvn test

mvn clean verify
```

```
mvn dependency:tree

mvn org.owasp:dependency-check-maven:check

mvn org.pitest:pitest-maven:mutationCoverage

mvn clean verify sonar:sonar -Dsonar.login=YOUR_SONAR_TOKEN
```

Each of these commands has a specific function:

- **clean** clears the previous build state.
- **compile** validates the compilation of the main code.
- **test** executes the tests.
- **verify** executes the complete checks of the Maven cycle.
- **dependency:tree** helps to visualize transitive dependencies.
- **dependency-check:check** scans for vulnerable dependencies.
- **mutationCoverage** measures the robustness of tests via mutation.
- **sonar:sonar** sends the results to SonarQube.

2.17 Maven commands for this phase

Lab 1 reminds us that GitHub Actions was introduced very early in DigiBank to prepare for the industrialization of build, testing, and reproducibility. Workshop 2 extends this logic by adding quality and security to the pipeline.

Create the file `.github/workflows/digibank-quality.yml`:

```
name : DigiBank Quality Pipeline

on :

  push :

    branches : [ "main" , "master" , "develop" ]

  pull_request :
```

```
branches : [ "main" , "master" , "develop" ]

jobs :

  build-and-test :

    runs-on : ubuntu-latest

    steps :
-   name : Checkout repository
        uses : actions/checkout@v4

-   name : JDK 17 Setup
        uses : actions/setup-java@v4
        with :
            distribution : temurin
            Java version : 17
            cache : maven

-   name : Build and test
        run : mvn clean verify

-   name : Run OWASP Dependency-Check
        run : mvn org.owasp:dependency-check-maven:check

-   name : Upload Dependency-Check report
```

```

    uses : actions/upload-artifact@v4

    with :

      name : dependency-check-report

      path : "**/target/dependency-check-report.html"

- name : Run PITest

  run : mvn org.pitest:pitest-maven:mutationCoverage

- name : Upload PITest reports

  uses : actions/upload-artifact@v4

  with :

    name : pitest-reports

    path : "**/target/pit-reports"

```

This pipeline checks out the code, installs Java 17, rebuilds DigiBank, runs the tests, scans for dependencies, and produces PITest reports as downloadable artifacts. In a practical setting, this version is already very useful, as it makes the controls observable even without opening each report locally.

2.18 Variant GitHub Actions with SonarQube Cloud or external server

If you want to integrate SonarQube or SonarCloud into GitHub Actions, you must store the token in GitHub secrets. This is consistent with the course's security approach, which discourages hard-coding secrets in configuration files.

Add a job or step like this:

```

- name: SonarQube analysis

```

```
run: mvn clean verify sonar:sonar -Dsonar.token=${{
secrets . SONAR_TOKEN }} }

approx:

SONAR_HOST_URL: http://localhost:9000
```

In the case of a self-hosted SonarQube server, this server must be accessible from the GitHub Actions environment, which is not the case for a localhost instance on your machine. In practice, for the lab, it is often simpler to keep SonarQube local for classroom demonstrations and to use GitHub Actions primarily for building, testing, dependency checking, and pitesting.

2.19 Final Local Verification

At the end of this phase, the student should be able to:

- open DigiBank in IntelliJ IDEA;
- Reload all Maven modules;
- rebuild the project without errors;
- Launch SonarQube locally;
- perform a SonarQube scan via Maven;
- produce an OWASP Dependency-Check report;
- produce a PITest report;
- Push the code to GitHub with a ready-to-run analysis workflow.

A complete sequence of local commands can therefore be presented as follows:

```
CD digibank-parent

mvn clean install

mvn dependency:tree
```

```
mvn org.owasp:dependency-check-maven:check
-DnvdApiKey=YOUR_NVD_API_KEY

mvn org.pitest:pitest-maven:mutationCoverage

docker network create digibank-net

docker volume create sonarqube_data

docker volume create sonarqube_logs

docker volume create sonarqube_extensions

docker run -d \

  --name sonarqube \

  --network digibank-net \

  -p 9000:9000 \

  -v sonarqube_data:/opt/sonarqube/data \

  -v sonarqube_logs:/opt/sonarqube/logs \

  -v sonarqube_extensions:/opt/sonarqube/extensions \

  sonarqube:lts-community

mvn clean verify sonar:sonar \

  -Dsonar.projectKey=digibank-parent \

  -Dsonar.projectName= "DigiBank Parent" \

  -Dsonar.host.url=http://localhost:9000 \
```

```
-Dsonar.login=YOUR_SONAR_TOKEN
```

This sequence provides a solid foundation for the rest of Workshop 2, where we can begin to effectively identify the vulnerabilities of the DigiBank project based on the results provided by the tools.

2.20 Expected result at the end of the game

At the end of Part 2, the student should not yet have fixed DigiBank's vulnerabilities, but they should have prepared the entire analysis framework. They have an open and rebuilt project, a Maven parent instance enhanced with the necessary control plugins, a working SonarQube server, reproducible dependency scans, a mutation testing measure, and a GitHub Actions pipeline ready to automate these checks.

In other words, the DigiBank project here goes from the status of a functional application from workshop 1 to that of an instrumented system for static analysis, which corresponds exactly to the pedagogical logic of the course: prevent early, observe objectively, then correct in an reasoned manner.

Part 3: Detailed identification of static vulnerabilities in DigiBank's source code

3.1 Role of this phase in the workshop

At this stage of workshop 2, the student has a rebuilt DigiBank project, equipped for static analysis and accompanied by tools capable of generating alerts, metrics, and reports. However, the pedagogical approach of this phase is not to mechanically copy the results of SonarQube, OWASP Dependency-Check, or PITest. Rather, it focuses on learning to interpret DigiBank as a risk environment, identifying weaknesses within its structure that make the application more susceptible to certain vulnerabilities, and then understanding why these weaknesses must be addressed.

This third part therefore constitutes the intellectual core of workshop 2. It is here that the student moves from technical instrumentation to security interpretation. After preparing the SAST environment, they must now examine the Java code, DTOs, services, controllers, exceptions, API responses, and configuration files in order to identify the main categories of static vulnerabilities targeted by the course.

The DigiBank project is particularly well-suited to this stage, as its first functional version was intentionally built as a clean, but not yet hardened, educational foundation. This means that the application works, compiles, exposes its APIs, and passes its main tests, while still containing a set of realistic weaknesses: insufficiently strict validations, overly verbose error messages, excessive exposure of certain data, secrets visible in the configuration, dependencies to monitor, and business logic that is sometimes insufficiently defined.

3.2 General logic for identifying vulnerabilities

In a SAST workshop, not all vulnerabilities appear in the same form. Some are directly visible in the code, for example, a password written in plain text in an `application.yml` file, an overly descriptive exception, or an output DTO exposing sensitive data. Others are more subtle: weak validation, an insufficiently controlled critical business operation, a call structure that makes access controls difficult to implement, or a service that reveals exploitable information through its error messages.



For this reason, identifying vulnerabilities in DigiBank must follow a structured method. Students should not simply ask, “Is there a bug?” but rather: what data enters the system, how is it validated, what information is extracted, what business assumptions are implicit, what messages are exposed, what external components are used, and which parts of the system would be easy or difficult to further secure in subsequent workshops. This approach to analyzing the project corresponds exactly to the definition of static analysis given in Part 1 of Lab 2.

In this section, the analysis will be conducted module by module, in order to adhere to the DigiBank vertical architecture established in Lab 1. The common module will be examined for its exception handling and API response mechanisms. The customer-module, account-module, and transfer-module modules will be analyzed in terms of validations, data exposure, business rules, and error messages. Finally, the digibank-web module and the Maven files will be examined for configuration weaknesses and dependencies.

3.3 Types of vulnerabilities actually searched for in DigiBank

In this version of DigiBank, the static vulnerabilities to identify fall into several categories consistent with the course material. The first concerns the validation of incoming data. Validation that is present but too permissive only provides superficial security. The second concerns error messages and exceptions. A banking application should not reveal too much information to a customer or a potential attacker about the existence of a resource, the status of an account, or the application's structure.

The third category encompasses the exposure of sensitive data in output DTOs or API responses. The fourth concerns secrets, credentials, and configuration parameters visible in versioned files. The fifth relates to dependencies and external components, which can introduce known vulnerabilities regardless of the quality of the internal code. Finally, a sixth category addresses the fragility of business rules, particularly in critical areas such as account creation and transfer execution.

The student must therefore be able to distinguish between a directly exploitable vulnerability and a structural signal of fragility. This distinction is important. SAST does not always prove the actual exploitation of a flaw, but it does indicate points where the risk is more likely, where the attack surface is expanded, or where future remediation will be more difficult if nothing is fixed now.

3.4 Reading the common module

The common module plays a cross-cutting role in DigiBank. It centralizes API responses and certain common exceptions, making it a strategic point of observation in a SAST framework. A weakness introduced here can easily propagate to all other modules, as all controllers and services subsequently rely on these shared components.

3.4.1 ApiResponse Class

Create or open the file:

common-module/src/main/java/com/m2ibank/common/api/ApiResponse.java

Extract

```
audience class ApiResponse < T > {  
  
    private boolean success ;  
  
    private String message ;  
  
    private T data ;  
  
    private LocalDateTime timestamp ;  
  
}
```

Vulnerability

The response structure is too permissive if it carries objects containing sensitive data without restriction.

Explanation

The ApiResponse class is not inherently vulnerable. It becomes problematic if the data field contains overly rich or unfiltered objects, and if the message field relays overly detailed information from services. In DigiBank, this risk is significant because the same response envelope is used by several business domains handling personal and financial data. Students must therefore understand that security depends not only on code syntax, but also on serialization conventions and the governance of exposed information.

Correction

Retain the class as a technical envelope, but require that only strictly controlled output DTOs be placed in data, and ensure that the message remains generic on the client side.

3.4.2 ResourceNotFoundException

Create or open the file:

common-module/src/main/java/com/m2ibank/common/exception/ResourceNotFoundException.java

Extract

```
audience      class      ResourceNotFoundException      extends
RuntimeException {

    audience ResourceNotFoundException ( String message ) {

        great (message) ;

    }

}
```

Vulnerability

Possible transmission of an overly descriptive error message to the REST layer.

Explanation

This exception freely accepts text provided by business services. However, in DigiBank, several services construct messages of the type “Customer not found with id X” or “Account not found with id Y”. If these messages are sent directly to the API, they facilitate resource enumeration and provide clues about the system's internal structure. Therefore, SAST should lead students to consider exceptions as information carriers, and not simply as flow control tools.

Correction

Introduce a `GlobalExceptionHandler`, normalize error responses exposed to the client, and reserve detailed messages for application logs.

3.4.3 BusinessException

Create or open the file:

`common-module/src/main/java/com/m2ibank/common/exception/BusinessException.java`

Extract

```
audience class BusinessException extends RuntimeException {  
    audience BusinessException ( String message ) {  
        great (message) ;  
    }  
}
```

Vulnerability

Same problem as for `ResourceNotFoundException`, with the added risk of disclosure of the system's business state.

Explanation

In a banking application, business errors are critical because they can reveal the status of an account, the presence of an already registered customer, or the precise reason for a rejected transfer. An overly verbose business exception is therefore not insignificant. It can be used by an attacker to confirm certain assumptions about the data or the system's behavior.

Correction

Maintain the distinction between business error and technical error, but filter the level of detail visible from the API.

3.5 Reading the customer module

The customer-module manages DigiBank's logical input: customers. This module is particularly interesting for static analysis because it handles several sensitive personal data points, exposes REST endpoints, and relies on declarative validations that may seem sufficient at first glance but are not actually so.

3.5.1 DTO CustomerRequest

Create or open the file:

customer-module/src/main/java/com/m2ibank/customer/dto/CustomerRequest.java

Extract

```
audience class CustomerRequest {

@NotBlank ( message ) = "Full name is required "

@ Size ( max = 120 )

    private String fullName ;

@NotBlank ( message ) = "Email is required "

@ Email ( message) = "Email format is invalid "

    private String email ;

@NotBlank ( message ) = "Phone number is required "

@ Size ( max = 30 )
```

```
private String phoneNumber ;

@NotBlank ( message ) = "National ID is required "

@ Size ( max = 30 )

private String nationalId ;

}
```

Vulnerability

Insufficient validation on certain critical fields, notably `phoneNumber` and `nationalId`.

Explanation

The DTO gives an impression of security through the use of Jakarta Validation annotations. However, `@Size` only checks the length, not the actual structure of the data. There is no guarantee that a phone number follows an acceptable format, nor that a national identifier complies with the expected rule. In a banking system, this weakness is significant because it allows inconsistent or poorly validated data to enter. One of the educational objectives of SAST is precisely to demonstrate that the presence of validations does not automatically equate to good validation.

Correction

Add shape constraints with `@Pattern`, or set up dedicated business validators for sensitive fields.

3.5.2 DTO CustomerResponse

Create or open the file:

`customer-module/src/main/java/com/m2ibank/customer/dto/CustomerResponse.java`

Extract

```
audience class CustomerResponse {  
  
    private Long id ;  
  
    private String fullName ;  
  
    private String email ;  
  
    private String phoneNumber ;  
  
    private String nationalId ;  
  
    private LocalDateTime createdAt ;  
  
}
```

Vulnerability

Excessive exposure of sensitive data in API responses.

Explanation

The fact that nationalId is always returned in the standard response poses a data minimization problem. Even though Workshop 1 hasn't yet introduced all the authentication and authorization logic, SAST should lead students to realize that an API should never indiscriminately expose all the attributes of a sensitive entity. This weakness also directly prepares students for later discussions on access control and DAST.

Correction

Create a more restricted output DTO, hide or remove sensitive fields not needed for client use of the API.

3.5.3 CustomerService.createCustomer

Create or open the file:

customer-module/src/main/java/com/m2ibank/customer/service/CustomerService.java

Extract

```
audience Customer Response createCustomer ( CustomerRequest
request) {

    customerRepository . findByEmail ( request.getEmail ( ) ).
ifPresent (c -> {

        throw new BusinessException ( "A customer with this email
already exists" );

    });

    customerRepository .    findByPhoneNumber    (
request.getPhoneNumber ( ) ). ifPresent (c -> {

        throw new BusinessException ( "A customer with this phone
number already exists" );

    });

    Customer customer = new Customer (

        request.getFullName ( ) ,

        request.getEmail ( ) ,

        request.getPhoneNumber ( ) ,

        request.getNationalId ( )

    );

    Customer saved = customerRepository . save (customer);
```

```
return mapToResponse (saved) ;  
}
```

Vulnerability

Business messages that are too explicit and confirm the existence of information in the database.

Explanation

The errors “email already exists” and “phone number already exists” are functionally useful, but they also reveal that data is already stored. In an exposed system, this behavior can allow the enumeration of email addresses or phone numbers. SAST (System Application Support Testing) should lead students to understand that a good functional experience should not result in a preventable data leak.

Correction

Use more neutral messages on the client side, and keep the precise details in logs or in internal layers.

3.5.4 CustomerService.getCustomerById

Extract

```
audience Customer Response getCustomerById ( Long id) {  
    Customer customer = customerRepository . findById (id)  
    . orElseThrow (() -> new ResourceNotFoundException ( "Customer  
not found with id " + id));  
    return mapToResponse (customer);  
}
```

Vulnerability

Error message facilitating the enumeration of identifiers.

Explanation

The requested identifier is included in the error message. While this may seem trivial, this type of feedback enhances a malicious user's understanding of how resources are referenced and verified. In the spirit of SAST, this isn't necessarily an isolated critical vulnerability, but rather a pattern in the code that increases the system's observable surface area.

Correction

Reduce the displayed message to the bare minimum, for example "Customer not found".

3.6 Reading the account-module

The account module represents a second level of sensitivity, as it handles bank accounts, balances, and customer-related identifiers. In a SAST approach, this module must be viewed not only as functional code, but also as a critical area from the perspective of financial data and business operations.

3.6.1 DTO AccountRequest

Create or open the file:

account-module/src/main/java/com/m2ibank/account/dto/AccountRequest.java

Extract

```
audience class AccountRequest {  
  
@NotNull ( message ) = "Customer ID is required "  
  
    private Long customerId ;  
  
}
```

```

@NotNull ( message ) = "Account type is required "

    private Account Type accountType ;

@NotNull ( message ) = "Initial balance is required" )

@DecimalMin ( value ) = "0.00" , inclusive = true ,

        message = "Initial balance must be greater than or
equal to zero" )

    private BigDecimal initialBalance ;

}

```

Vulnerability

Only partial validation of the initial amount.

Explanation

The constraint prohibits negative values, which is useful, but does not constitute complete business validation. The system imposes neither a control strategy on accuracy, nor a functional rule on account type, nor any additional context-dependent limits. SAST should highlight an important point here: a correct numerical rule is not necessarily a sufficient business rule.

Correction

Keep @DecimalMin, but complete the validation in the service with explicit business rules.

3.6.2 AccountService.createAccount

Create or open the file:

account-module/src/main/java/com/m2ibank/account/service/AccountService.java

Extract

```
audience AccountResponse createAccount ( AccountRequest request)
{
    customerService . getCustomerEntityById (
request.getCustomerId ( ) );

    String accountNumber = generateAccountNumber ( );

    while ( accountRepository.findByAccountNumber (accountNumber)
.isPresent ( ) ) {
accountNumber = generateAccountNumber ( );
}

    Account account = new Account (
accountNumber,

    request . getInitialBalance ( ),

    request.getAccountType ( ) ,

    request.getCustomerId ( )

);

    Account saved = accountRepository . save (account);

    return mapToResponse (saved);
}
```

Vulnerability

Critical business logic is present, but not very explicit in terms of security and consistency control.

Explanation

The code correctly verifies the client's existence and avoids the trivial collision of account numbers, but it doesn't clearly express other structural controls. Account creation depends on an identifier provided by the API client, which, in a real-world system, would require additional safeguards. The SAST course should guide students to identify this type of strong reliance on raw IDs in business services.

Correction

Make controls more explicit, document business assumptions, and prepare a clearer authorization logic for future developments.

3.6.3 AccountService.getAccountEntityById

Extract

```
audience Account getAccountEntityById ( Long id) {  
  
    return accountRepository . findById (id)  
  
    . orElseThrow (() -> new ResourceNotFoundException ( "Account  
not found with id " + id));  
  
}
```

Vulnerability

Same pattern of information leakage as in the client module.

Explanation

The repetition of the same detailed exception pattern across multiple modules demonstrates that the weakness is not accidental, but structural. SAST is useful precisely for this type of cross-sectional view: it's not just an isolated message, but a coding convention that needs to be corrected at its root.

Correction

Standardize the error management strategy across the entire project.

3.6.4 debitAccount

Extract

```
audience void debitAccount ( Account account, BigDecimal amount)
{
    if ( account . getBalance (). compareTo (amount) < 0 ) {
        throw new BusinessException ( "Insufficient balance for
transfer" );
    }

    account . setBalance ( account.getBalance ( ) . subtract
(amount));

    accountRepository . save (account);
}
```

Vulnerability

Business message revealing account status and sensitive logic scattered throughout.

Explanation

The message confirms that the targeted account does not have sufficient funds. In a banking context, this information is significant. Furthermore, the method is involved in a critical

operation whose transactional robustness must be clearly visible. SAST not only flags syntax errors or dangerous API usage; it also draws attention to areas of code where sensitive operations are coded in a way that is either insufficiently robust or overly verbose.

Correction

Make the message more generic and clearly encapsulate financial operations within a controlled transactional logic.

3.7 Reading the transfer module

The transfer module is probably the most business-critical part of DigiBank. It handles transfers, and therefore changes in account balances, with direct impacts on the system's financial integrity. This module should be examined with particular attention in Workshop 2.

3.7.1 DTO TransferRequest

Create or open the file:

transfer-module/src/main/java/com/m2ibank/transfer/dto/TransferRequest.java if it conforms to the expected implementation of TP1.

Typical extract

```
audience class TransferRequest {  
  
    @NotNull ( message ) = "Source account is required" )  
  
        private Long sourceAccountId ;  
  
  
    @NotNull ( message ) = "Destination account is required" )  
  
        private Long destinationAccountId ;  
  
}
```

```

@NotNull ( message ) = "Amount is required "

@DecimalMin ( value ) = "0.01" , inclusive = (true )

    private BigDecimal amount ;

}

```

Vulnerability

Incomplete validation for a critical financial transaction.

Explanation

The amount is capped at the minimum, but there's still no guarantee that the source and destination accounts are separate, that the transaction is contextually authorized, or that more detailed business rules are followed. The SAST (Student Support and Analysis Tool) should help students see that the annotations only cover part of the problem.

Correction

Add strong business checks to the transfer service, particularly regarding the consistency of the accounts involved.

3.7.2 Transfer Service

Typical extract

```

audience TransferResponse transfer ( TransferRequest request) {

    Account source = accountService . getAccountEntityById (
request.getSourceAccountId ( ) );

    Account destination = accountService . getAccountEntityById (
request.getDestinationAccountId ( ) );

```

```
accountService . debitAccount ( source, request.getAmount (
));

accountService . creditAccount ( destination,
request.getAmount ( ));

Transfer transfer = new Transfer (...);

Transfer savedTransfer = transferRepository . save
(transfer);

return mapToResponse (savedTransfer);
}
```

Vulnerability

Direct use of account credentials in a critical operation, without sufficient visible control over the security context.

Explanation

The service relies on two identifiers provided by the request to manage a financial transaction. In a real-world application, this would require a rigorous authorization policy and a thorough understanding of the user context. While Workshop 1 was not intended to introduce complete security, Workshop 2 should lead students to recognize that this structure reveals a lack of oversight of critical business logic.

Correction

More strictly define the transfer logic, add consistency checks and prepare for the future introduction of robust access controls.

3.8 Reading the digibank-web and configuration files

Static analysis should never be limited to Java classes. In DigiBank, a significant portion of the risk also lies in the configuration. Lab 1 intentionally introduced some simple parameters to facilitate project startup, which now provides excellent material for analysis in Lab 2.

3.8.1 application.yml

Create or open the file:

digibank-web/src/main/resources/application-dev.yml

Extract

```
spring :  
  
  datasource :  
  
    url : jdbc:postgresql://localhost:5432/digibankdb  
  
    username : digibank  
  
    password : digibank123  
  
    driver-class-name : org.postgresql.Driver  
  
  jpa :  
  
    hibernate :  
  
      ddl-auto : update  
  
    show-sql : true
```

Vulnerability

Secrets exposed in plain sight, overly permissive development configuration, and potentially overly verbose logging.

Explanation

The database password is displayed in plain text in a versioned file. The `ddl-auto: update` mode simplifies the teaching process but is not desirable as a long-term convention. Finally, `show-sql: true` can expose query logs and therefore sensitive technical information. The SAST (Student Application Support Team) should enable students to understand that the risk comes not only from the business logic but also from the settings surrounding the application.

Correction

Externalize secrets, separate Spring profiles, limit log verbosity and reserve certain configurations for the development environment only.

3.8.2 Swagger / OpenAPI

Extract

```
springdoc :  
  
  api-docs :  
  
    path : /api-docs  
  
  swagger-ui :  
  
    path : /swagger-ui.html
```

Vulnerability

Increased exhibition space through interactive documentation.

Explanation

Enabling Swagger is excellent from a pedagogical standpoint and functionally useful. However, in a SAST context, it must also be considered as a potential source of exposure. A

publicly documented API facilitates testing and mapping. The goal here is not to condemn Swagger, but to learn how to reason about its use in different environments.

Correction

Restrict access to Swagger to certain profiles or environments, and make this choice explicit in the project's security strategy.

3.9 Reading Maven Dependencies

The SAST scope of Lab 2 also includes certain Maven dependency definitions, because a vulnerable library weakens the system even if the team's code is well-written. Lab 2 specifically prepared this analysis using OWASP Dependency-Check and `dependency:tree`.

3.9.1 Example of a dependency to monitor

Extract

```
< dependency >
    < groupId > org.springdoc </ groupId >
        < artifactId > springdoc-openapi-starter-webmvc-ui </
artifactId >
        < version > 2.6.0 </ version >
</dependency>
```

Vulnerability

Version to monitor from the point of view of obsolescence or possible CVEs.

Explanation



An explicitly versioned dependency isn't necessarily vulnerable, but it should be considered an attack surface element. Workshop 2 teaches you how to complement code analysis with an examination of embedded components. The OWASP Dependency-Check report will then confirm whether certain versions need updating or justification.

Correction

Analyze dependency reports, update relevant components as needed, and document version decisions.

3.10 Expected student method for this part

At this stage of the workshop, students must not simply list suspicious lines of code. They must adopt a structured analytical approach. For each vulnerability identified, they must be able to specify the relevant file, quote the pertinent excerpt, name the vulnerability or weakness, explain its potential impact within the DigiBank context, and then propose a fix consistent with the project's architecture. This requirement is entirely consistent with the pedagogical approach of Workshop 2, which emphasizes reasoned interpretation rather than the raw collection of alerts.

A good report can therefore take the form of a table or a summary sheet for each vulnerability, with the following sections: module, file, extract, type of vulnerability, estimated severity, probable impact, tool that can be used to report it, and expected remediation. This structure naturally prepares the way for the following section, which focuses on effective remediation measures.

3.11 Expected result at the end of part 3

At the end of this third part, the student should be able to analyze DigiBank as a project to be statically audited, and no longer simply as an application to be run. They should be able to recognize code patterns that indicate weak validation, excessive information exposure, dangerous or naive use of exceptions, an insufficiently secure configuration, or a potentially risky dependency.

He must also be able to justify why these points constitute vulnerabilities, or at least significant weaknesses, within a DevSecOps framework. In other words, it's no longer simply a matter of identifying what "could be improved," but of understanding what



expands DigiBank's risk surface and why early remediation is preferable to late remediation. This understanding directly prepares the ground for the next phase, where the objective will be to transform vulnerability identification into a structured remediation process.

Part 4: Detailed remediation of identified vulnerabilities

4.1 Role of this phase

At this stage of Workshop 2, DigiBank has already been instrumented for static analysis, the monitoring tools have been configured, and the initial results from SonarQube, OWASP Dependency-Check, and PITest have revealed several categories of weaknesses. The objective is no longer simply to observe the alerts, but to move on to the most important work of the workshop: the reasoned remediation of the vulnerabilities and weaknesses detected in the source code and in certain project configurations.

This phase is central to the DevSecOps approach of the course. An alert only has educational value if it leads to an understanding of the risk, an appropriate remediation, and a clear verification of the resulting improvement. In DigiBank, this involves reworking certain classes, YAML files, and some Maven dependencies to transform a purely functional foundation into a more robust, better-controlled one that is more compatible with future DAST frameworks, containers, and security pipelines.

Remediation should never be approached as a mechanical series of patches. Each fix must remain consistent with the project's modular architecture, not break the expected

business behavior, and be readable for the student, who must understand not only what has been changed, but also why this change effectively reduces the risk. For this reason, each vulnerability will be addressed here according to the same structure: initial vulnerable code, corrected code, explanation of the fix, verification commands, and then the expected impact on SonarQube, Dependency-Check, and PITest.

4.2 Remediation of sensitive information exposure in the application configuration

One of the first weaknesses visible in the initial version of DigiBank concerns the direct exposure of database credentials in the configuration files. Lab 1 itself already indicated that this configuration was acceptable only for initial pedagogical simplicity, but that it was not a sound security practice. In the context of a SAST workshop, this situation must now be corrected, as it falls precisely into the category of secrets visible in the configuration.

Vulnerable initial code

File digibank-web/src/main/resources/application-dev.yml:

```
spring :  
  
  application :  
  
    name : digibank  
  
  datasource :  
  
    url : jdbc:postgresql://localhost:5432/digibankdb  
  
    username : digibank  
  
    password : digibank123  
  
    driver-class-name : org.postgresql.Driver  
  
  jpa :  
  
    hibernate :
```

```
ddl-auto : update

show-sql : true
```

In this version, the database password and login user are directly stored in the repository. While the purpose here is educational, the coding pattern remains problematic because it normalizes a dangerous practice and could, in a real-world context, lead to a leak of secrets or misuse of the model by the student.

Code corrected

File digibank-web/src/main/resources/application.yml:

```
spring :

  application :

    name : digibank

  profiles :

    active : dev

  jpa :

    hibernate :

      ddl-auto : validate

    show-sql : false

    open-in-view : false

    properties :

      hibernate :

        SQL format : true

      jdbc :
```

```
    time zone : UTC

thymeleaf :

    cache : true

SQL :

    init :

        mode : never

server :

    port : 8080

    error :

        include-message : never

        include-binding-errors : never

        include-stacktrace : never

springdoc :

    api-docs :

        enabled : false

    swagger-ui :

        enabled : false

logging :

    level :
```

```
root : INFO

org.springframework.web : INFO

org.hibernate.SQL : INFO

org.hibernate.type.descriptor.sql.BasicBinder : INFO

error :

include-message : never

include-binding-errors : never

include-stacktrace : never
```

File digibank-web/src/main/resources/application-dev.yml:

```
spring :

  datasource :

    url : ${SPRING_DATASOURCE_URL}

    username : ${SPRING_DATASOURCE_USERNAME}

    password : ${SPRING_DATASOURCE_PASSWORD}

    driver-class-name : org.postgresql.Driver

  jpa :

    hibernate :

      ddl-auto : validate

    show-sql : false

    open-in-view : false

    properties :
```

```

hibernate :

    SQL format : true

    jdbc :

        time zone : UTC


flyway :

    enabled : true

    baseline-on-migrate : true

    locations : classpath:db/migration/postgresql

```

In an even cleaner approach, the student can create a local, unversioned file, for example .env

```

SPRING_DATASOURCE_URL
=jdbc:postgresql://localhost:5432/digibankdb

SPRING_DATASOURCE_USERNAME =digibank

SPRING_DATASOURCE_PASSWORD =digibank123

SONAR_TOKEN = YOUR_SONAR_TOKEN

NVD_API_KEY = YOUR_NVD_API_KEY

SONAR_HOST_URL =http://localhost:9000

```

or directly define environment variables in your shell or in the IntelliJ runtime configuration.

```

$env          :          SPRING_DATASOURCE_URL          =
"jdbc:postgresql://localhost:5432/digibankdb"

$env : SPRING_DATASOURCE_USERNAME = "digibank"

```

```
$env : SPRING_DATASOURCE_PASSWORD = "digibank123"

mvn clean verify

mvn spring-boot:run -pl digibank-web
```

This allows for simple local execution without teaching the bad habit of storing secrets in the Git repository.

Explanation of the correction

The fix relies on a clear separation between versioned configuration and sensitive values. Spring Boot allows referencing environment variables with a simple syntax, making the application compatible with local execution, Docker, GitHub Actions, or a deployment server, without hardcoding identifiers in shared files.

This fix doesn't make DigiBank fully compliant with a production security level, but it's a significant improvement. It reduces the likelihood of accidental secret exposure, prepares for future integration with Docker Compose and GitHub Actions, and aligns the project with one of the course's core messages: a configuration file should not act as a vault.

The student must verify that the application starts correctly, connects properly to PostgreSQL, and does not fail during Spring context initialization. It is also important to check that the Git repository no longer contains an explicit password in application.yml.

Impact on tools

In SonarQube, this remediation reduces signals related to secrets or sensitive information exposed in configuration files and improves the overall perception of project security. The exact effect depends on the rules enabled, but the configuration quality is objectively better.

OWASP Dependency-Check is not directly impacted by this change, as the tool works on dependencies and not configuration secrets. However, PITest is also not directly affected, unless new integration tests are added around application startup with environment variables.

4.3 Remediation of insufficient input validations on customer data

In the initial version of the project, the Data Transfer Objects (DTOs) already used certain validation annotations such as `@NotBlank`, `@Email`, and `@Size`, which provided a good foundation for learning. However, in a banking context, these validations remained too permissive for certain fields like name, phone number, and national identification number. A thorough static analysis must specifically address this type of weakness, as incomplete validations facilitate the introduction of inconsistent, malformed, or potentially dangerous data into application flows.

Vulnerable initial code

File

customer-module/src/main/java/com/m2ibank/customer/dto/CustomerRequest.java:

```
package com.m2ibank.customer.dto ;

import jakarta.validation.constraints.Email ;
import jakarta.validation.constraints.NotBlank ;
import jakarta.validation.constraints.Size ;

audience class CustomerRequest {

    @NotBlank ( message ) = "Full name is required "

    @ Size ( max = 120 )

        private String fullName ;

    @NotBlank ( message ) = "Email is required "

    @ Email ( message ) = "Email format is invalid "
```

```

    private String email ;

    @NotBlank ( message ) = "Phone number is required "

    @ Size ( max = 30 )

    private String phoneNumber ;

    @NotBlank ( message ) = "National ID is required "

    @ Size ( max = 30 )

    private String nationalId ;

    // getters and setters

}

```

The problem here isn't the complete absence of validation, but its insufficient level. A phone number containing arbitrary characters or a national identifier with an absurd structure could still be accepted. However, in DigiBank, this data plays a crucial business role and must be subject to stricter controls.

Code corrected

```

package com.m2ibank.customer.dto ;

import jakarta.validation.constraints.Email ;

import jakarta.validation.constraints.NotBlank ;

import jakarta.validation.constraints.Pattern ;

```

```

import jakarta.validation.constraints.Size ;

audience class CustomerRequest {

    @NotBlank ( message ) = "Full name is required "

    @ Size ( min = 3 , max = 120 , message = "Full name must contain
between 3 and 120 characters" )

    @Pattern (

        regexp = "^[A-Za-zÀ-ÿ' -]+$" ,

        message = "Full name contains invalid characters"

    )

    private String fullName ;

    @NotBlank ( message ) = "Email is required "

    @ Email ( message ) = "Email format is invalid "

    @ Size ( max = 150 , message = "Email must not exceed 150
characters" )

    private String email ;

    @NotBlank ( message ) = "Phone number is required "

    @Pattern (

        regexp = "^ \\ +?[0-9]{8,15}$" ,

```

```

        message = "Phone number format is invalid"
    )

    private String phoneNumber ;

    @NotBlank ( message ) = "National ID is required "
    @Pattern (

        regexp = "^[A-Z0-9_-]{5,30}$" ,

        message = "National ID format is invalid"
    )

    private String nationalId ;

    audience String getFullName () {

        return fullName;
    }

    audience void setFullName ( String fullName ) {

        this.fullName = fullName == null ? null : fullName.trim (
    ) ;
    }

    audience String getEmail () {

        return email;
    }

```

```
}

    audience void setEmail ( String email ) {

        this . email = email == null ? null : email . trim ().
toLowerCase ();
    }

    audience String getPhoneNumber () {

        return phoneNumber;
    }

    audience void setPhoneNumber ( String phoneNumber ) {

        this . phoneNumber = phoneNumber == null ? null :
phoneNumber.trim ( ) ;
    }

    audience String getNationalId () {

        return nationalId;
    }

    audience void setNationalId ( String nationalId ) {
```

```
        this . nationalId = nationalId == null ? null :  
nationalId . trim (). toUpperCase ();  
}  
}
```

Explanation of the correction

Remediation improves validation at two levels. First, stronger format constraints are added with `@Pattern`, which limits the accepted characters and enforces a structure compatible with the expected business use. Second, setters normalize certain data, such as lowercase email addresses or uppercase national identifiers, which reduces inconsistencies and improves the overall quality of the records.

This correction clearly illustrates an important idea of SAST: a weakness can exist even when the code compiles, runs, and already contains validation annotations. What is evaluated here is not simply the presence of a validation, but its actual relevance to the risk level of the domain being addressed. In a banking application, accepting overly flexible formats for customer identifiers is a logical and documented weakness in the code.

Verification commands

```
mvn clean test  
  
mvn clean verify
```

The student can also verify the behavior via Swagger or with curl:

```
curl -X POST http://localhost:8080/api/customers \  
  
-H "Content-Type: application/json" \  
  
-d '{  
  
"fullName": "@@@",
```

```
"email": "bad-email",  
  
"phoneNumber": "abc123",  
  
"nationalId": "***"  
  
'
```

The request should now be rejected with an HTTP 400 response and a consistent validation message, without stack trace or unnecessary technical details. This test allows us to directly link the static remediation to an observable improvement in the API's input contract.

Impact on tools

SonarQube interprets this type of improvement favorably, as it strengthens code robustness and reduces the risks associated with poorly controlled inputs. Even if not all business validations result in an explicit Sonar rule, maintainability and perceived code quality improve.

OWASP Dependency-Check is not directly affected. However, PITest could be positively impacted if additional unit tests now check for invalid cases, as these tests will help kill more mutants on the field validation and transformation branches.

4.4 Remediation of insufficient input validations on customer data

Lab 1 had already introduced a `GlobalExceptionHandler`, which provided a very good foundation for preventing raw errors. However, in an initial training version, it's still common for some error responses to reveal too much information or fail to clearly distinguish between business cases and internal failures. This is problematic because overly verbose messages help an attacker better understand the application's internal structure, field names, or the detailed behavior of its processes.

Vulnerable initial code

```
@RestControllerAdvice
```

```

audience class GlobalExceptionHandler {

    @ExceptionHandler ( ResourceNotFoundException.class )

        audience ResponseEntity < ApiResponse <Object> >
handleNotFound ( ResourceNotFoundException ex ) {

        return ResponseEntity . status ( HttpStatus . NOT_FOUND )

        . body ( ApiResponse.failure ( ex.getMessage ( ) , null ) ) ;
    }

    @ExceptionHandler ( BusinessException.class )

        audience ResponseEntity < ApiResponse <Object> >
handleBusinessException ( BusinessException ex ) {

        return ResponseEntity . status ( HttpStatus.BAD_REQUEST )

        . body ( ApiResponse.failure ( ex.getMessage ( ) , null ) ) ;
    }

    @ExceptionHandler ( MethodArgumentNotValidException.class )

        audience ResponseEntity < ApiResponse <Object> >
handleValidationException ( MethodArgumentNotValidException ex )
    {

        String message = ex . getBindingResult ( ) . getFieldErrors
        ( )

        .stream ( )

```

```

    . findFirst ()

    .   map   (error -> error.getField ( ) + ": " +
error.getDefaultMessage ( ) )

    .   orElse ( "Validation error" );

        return ResponseEntity . status ( HttpStatus.BAD_REQUEST )

    .   body ( ApiResponse.failure ( message, null ) ) ;
}

@ExceptionHandler ( Exception.class )

    audience   ResponseEntity < ApiResponse <Object> >
handleGenericException ( Exception ex ) {

        return ResponseEntity . status ( HttpStatus .
INTERNAL_SERVER_ERROR )

    .   body ( ApiResponse.failure ( "An unexpected error occurred" ,
null ) ) ;
}

}

```

This version is already better than a raw stack trace, but it still needs improvement. The concatenation of the field name and message can expose more information than necessary, and the response structure doesn't clearly distinguish between validation contexts, business errors, and internal incidents in a more standardized way.

Code corrected

Create

the

class

common-module/src/main/java/com/m2ibank/common/api/ApiError.java:

```
package com.m2ibank.common.api;

import java.time.LocalDateTime;
import java.util.List;

public class ApiError {

    private boolean success ;

    private String message ;

    private List<String> details ;

    private LocalDateTime timestamp ;

    public ApiError () {

        this . timestamp = LocalDateTime. now () ;

        this.success = false ;

    }

    public ApiError (String message, List<String> details) {

        this () ;

        this.message = message ;

        this.details = details ;

    }

}
```

```

        public static ApiError of (String message, List<String>
details) {

            return new ApiError(message, details);

        }

        public boolean isSuccess () {

            return success ;

        }

        public String getMessage () {

            return message ;

        }

        public List<String> getDetails () {

            return details ;

        }

        public LocalDateTime getTimestamp () {

            return timestamp ;

        }

    }
}

```

Then replace the global manager with:

```

package com.m2ibank.web.exception;

```

```

import com.m2ibank.common.api.ApiError;

import com.m2ibank.common.exception.BusinessException;

import com.m2ibank.common.exception.ResourceNotFoundException;

import org.springframework.http.HttpStatus;

import org.springframework.http.ResponseEntity;

import
org.springframework.web.bind.MethodArgumentNotValidException;

import org.springframework.web.bind.annotation. ExceptionHandler
;

import org.springframework.web.bind.annotation.
RestControllerAdvice ;

import java.util.List;

@RestControllerAdvice

public class GlobalExceptionHandler {

    @ExceptionHandler (ResourceNotFoundException. class )

    public ResponseEntity<ApiError> handleNotFound
(ResourceNotFoundException ex) {

        return ResponseEntity. status ( HttpStatus.NOT_FOUND )

        .body(ApiError. of ( "Requested resource was not found" , List.
of (ex.getMessage()) ));
    }
}

```

```

    @ExceptionHandler (BusinessException. class )

    public ResponseEntity<ApiError> handleBusinessException
    (BusinessException ex) {

        return ResponseEntity. status ( HttpStatus.BAD_REQUEST )

        .body(ApiError. of ( "Business rule violation" , List. of
        (ex.getMessage())));
    }


    @ExceptionHandler (MethodArgumentNotValidException. class )

    public ResponseEntity<ApiError> handleValidationException
    (MethodArgumentNotValidException ex) {

List<String> details = ex.getBindingResult()

        .getFieldErrors()

        .stream()

        .map(error -> error.getDefaultMessage())

        .distinct()

        .toList();

        return ResponseEntity. status ( HttpStatus.BAD_REQUEST )

        .body(ApiError. of ( "Input validation failed" , details));
    }


    @ExceptionHandler (Exception. class )

    public ResponseEntity<ApiError> handleGenericException
    (Exception ex) {

```

```
        return ResponseEntity. status (
HttpStatus.INTERNAL_SERVER_ERROR )

        .body(ApiError. of ( "Internal server error" , List. of ( "The
operation could not be completed" ))) ;
    }
}
```

Explanation of the correction

The fix serves three purposes. First, it clearly separates application errors into a dedicated structure, making the response contract more stable and readable. Second, it reduces the amount of information exposed in public responses by avoiding unnecessary details about internal field names or root causes. Third, it facilitates better future traceability, as the application will be able to log more accurately internally without exposing everything to the client.

The idea is not to make the system so opaque as to render it unusable, but to limit the disclosure of technical clues. This distinction is important: good error handling provides the user or tester with sufficient information about the nature of the problem, without gratuitously giving them a detailed map of the backend's internal logic.

Verification commands

```
curl -X POST http://localhost:8080/api/transfers \

-H "Content-Type: application/json" \

-d '{

"sourceAccountId": 1,

"destinationAccountId": 1,

"amount": 1000,

"description": "Self transfer"

}'
```

```
curl http://localhost:8080/api/customers/999999
```

The student must verify that the answer contains no stack trace, no internal Java classes, no excessive technical detail, and that the JSON format remains consistent from one case to another.

Impact on tools

SonarQube can positively signal the reduction of certain brittle practices related to error handling and information exposure. Even when the improvement doesn't correspond to an explicit critical rule, the readability and robustness of exception handling clearly progress.

Dependency-Check is not affected. PITest can be improved if tests now check exception branches and responses returned by the global handler, as this increases the tests' ability to detect artificial modifications to HTTP messages or statuses.

4.5 Remediation of an incomplete business rule in the transfer service

The transfer service is one of DigiBank's most sensitive points. In its first version, it already prevented transfers from an account to itself and relied on insufficient balance control at the time of debit, which was an acceptable basis for workshop 1. However, in a more demanding SAST logic, several business controls remain insufficiently expressed: verification of a zero or negative amount at the service level, presence of an empty or poorly normalized description, and lack of strong explicitness of the business preconditions in the method itself.

Vulnerable initial code

```
@ Transactional

audience TransferResponse createTransfer ( TransferRequest
request) {

    if ( request.getSourceAccountId ( ). equals (
request.getDestinationAccountId ( ) )) {
```

```
        throw new BusinessException ( "Source and destination
accounts must be different" );
    }

    Account sourceAccount = accountService . getAccountEntityById
( request.getSourceAccountId ( ) );

    Account destinationAccount = accountService .
getAccountEntityById ( request.getDestinationAccountId ( ) );

    accountService . debitAccount ( sourceAccount,
request.getAmount ( ) );

    accountService . creditAccount ( destinationAccount,
request.getAmount ( ) );

    Transfer transfer = new Transfer (

        request . getSourceAccountId ( ),

        request . getDestinationAccountId ( ),

        request.getAmount ( ) ,

        request.getDescription ( )
    );

    Transfer saved = transferRepository . save (transfer);

    return mapToResponse (saved);
```

```
}
```

Even though the DTO already carries @DecimalMin validation, the business service remains too dependent on this external layer and does not explicitly revalidate its preconditions. However, from a defensive security perspective, critical rules must exist where the business decision is actually made.

Code corrected

```
@Transactional

    audience TransferResponse createTransfer ( TransferRequest
request) {

    if (request == null ) {

        throw new BusinessException ( "Transfer request must not
be null" );

    }

    if ( request.getSourceAccountId ( ) == null || request .
getDestinationAccountId ( ) == null ) {

        throw new BusinessException ( "Source and destination
accounts are required" );

    }

    if ( request.getSourceAccountId ( ). equals (
request.getDestinationAccountId ( ) ) ) {
```

```

        throw new BusinessException ( "Source and destination
accounts must be different" );
    }

    if ( request.getAmount ( ) == null || request . getAmount ( ).
signum ( ) <= 0 ) {

        throw new BusinessException ( "Transfer amount must be
greater than zero" );
    }

    String description = request . getDescription ( ) == null ? "
: request . getDescription ( ). trim ( );

    if ( description . length ( ) > 255 ) {

        throw new BusinessException ( "Transfer description is
too long" );
    }

    Account sourceAccount = accountService . getAccountEntityById
( request.getSourceAccountId ( ) );

    Account destinationAccount = accountService .
getAccountEntityById ( request.getDestinationAccountId ( ) );

```

```

        accountService . debitAccount ( sourceAccount,
request.getAmount ( ));

        accountService . creditAccount ( destinationAccount,
request.getAmount ( ));

Transfer transfer = new Transfer (

    request . getSourceAccountId (),

    request . getDestinationAccountId (),

    request.getAmount ( ) ,

description
);

Transfer saved = transferRepository . save (transfer);

return mapToResponse (saved);
}

```

Explanation of the correction

The solution involves judiciously replicating business logic safeguards within the service itself. In a sound architecture, the DTO acts as the first validation barrier, but the service shouldn't blindly assume that this barrier has always been successfully passed. More defensive code better protects the core business logic against indirect calls, poorly designed tests, future reuses, or controller changes.

This remediation also improves testability. By explicitly defining business preconditions in the core method, expected behaviors become more visible, easier to verify,

and more resilient to future changes. This is precisely the type of improvement that paves the way for both greater application robustness and better results under mutation testing.

Verification commands

```
mvn clean test

mvn org.pitest:pitest-maven:mutationCoverage
```

Possible manual tests:

```
curl -X POST http://localhost:8080/api/transfers \
  -H "Content-Type: application/json" \
  -d '{
    "sourceAccountId": 1,
    "destinationAccountId": 2,
    "amount": 0,
    "description": "Invalid transfer"
  }'
```

```
curl -X POST http://localhost:8080/api/transfers \
  -H "Content-Type: application/json" \
  -d '{
    "sourceAccountId": 1,
    "destinationAccountId": 1,
```

```
"amount": 1000,  
  
"description": "Invalid transfer"  
}'
```

The application must now clearly reject these cases, and the associated unit tests must prove that the business rejection branches are properly covered.

Impact on tools

SonarQube emphasizes clarifying business logic, reducing implicit assumptions, and improving code robustness. This can result in a decrease in certain reliability signals or a better overall assessment of service quality.

Dependency-Check is not affected. However, PITest can be greatly improved if tests explicitly target each of the newly introduced branches, as mutants involving comparisons, nullity checks, or amount conditions will have a greater chance of being eliminated.

4.6 Remediation of an obsolete or vulnerable dependency detected by OWASP Dependency-Check

Static analysis in DigiBank is not limited to the Java code written by the student. It also encompasses the third-party libraries used by the project. Lab 2 explicitly emphasizes the role of OWASP Dependency-Check and the idea that certain old or vulnerable dependencies should be considered system weaknesses, even when the business logic appears correct.

In an educational essay, it is perfectly legitimate to present a typical remediation approach for a dependency detected as obsolete or vulnerable in the project. The important point is not simply to update a version number, but to show the student how to link a scan report to a concrete fix in Maven.

Vulnerable initial code

Example of a dependency or outdated frozen version in the Maven parent:

```
< properties >

  < spring.boot.version > 3.3.2 </ spring.boot.version >

  < springdoc.version > 2.6.0 </ springdoc.version >

      < dependency-check.version > 10.0.4 </
dependency-check.version >

</ properties >
```

In practice, the exact vulnerability will depend on the timing of the scan and the CVE database used by the tool. The key takeaway for students is that the Dependency-Check report can flag a direct or transitive dependency that has become risky, and this information should be used within the Maven workflow.

Code corrected

Example of a well-reasoned update:

```
< properties >

  < spring.boot.version > 3.4.3 </ spring.boot.version >

  < springdoc.version > 2.6.0 </ springdoc.version >

      < dependency-check.version > 10.0.4 </
dependency-check.version >

</ properties >
```

Or, when targeted exclusion is necessary:

```
< dependency >

  < groupId > org.springframework.boot </ groupId >
```

```
< artifactId > spring-boot-starter-web </ artifactId >

< exclusions >

  < exclusion >

    < groupId > vulnerable.group </ groupId >

    < artifactId > vulnerable-artifact </ artifactId >

  </exclusion>

</ exclusions >

</dependency>
```

Then replace it with an explicitly declared corrected version:

```
< dependency >

  < groupId > vulnerable.group </ groupId >

  < artifactId > vulnerable-artifact </ artifactId >

  < version > fixed-version </ version >

</dependency>
```

Explanation of the correction

Remediation of a vulnerable dependency should always be guided by the report generated by the tool. It's not about blindly updating the entire Maven graph, but about identifying the components that are actually exposed, understanding whether they are direct or transitive, and then choosing between a version update, a targeted exclusion, or a library change if necessary.

In DigiBank, this approach is particularly important, as Workshop 4 will more broadly address the security of dependencies, Docker images, and delivery artifacts.

Workshop 2 already prepares participants for this by demonstrating that security depends not only on locally written code, but also on the third-party components included in the project.

Verification commands

```
mvn dependency:tree

mvn org.owasp:dependency-check-maven:check
-DnvdApiKey=YOUR_NVD_API_KEY

mvn clean verify
```

The student must then open the generated HTML report, compare the state before and after the update, and then verify that the problematic dependency no longer appears as vulnerable or that its severity level has been significantly reduced.

Impact on tools

The main impact concerns OWASP Dependency-Check, which should show a reduction in the number of CVEs detected or an improvement in the overall severity level of the report. This is the most directly affected indicator.

SonarQube is generally only indirectly impacted, either by an overall improvement in project quality or by rules related to certain libraries. PITest, on the other hand, is not directly affected, unless the dependency update requires supplementing or adapting certain behavioral tests.

4.7 Remediation of an obsolete or vulnerable dependency detected by OWASP Dependency-Check

PITest's role in this workshop is not to detect a security vulnerability in the traditional sense, but to demonstrate that seemingly secure code can remain poorly protected if tests fail to detect significant logical changes. In DigiBank, this observation is particularly important for business services like AccountService or TransferService, where a mutation in a comparison, subtraction, or business condition can sometimes persist if the tests are too superficial.

Let us take here the case of account debit, the business logic of which is sensitive and directly linked to the functional security of the banking application.

Initial code is fragile when compared to PITest

```
audience void debitAccount ( Account account, BigDecimal amount)
{
    if ( account . getBalance (). compareTo (amount) < 0 ) {
        throw new BusinessException ( "Insufficient balance for
transfer" );
    }

    account . setBalance ( account.getBalance ( ) . subtract
(amount));

    accountRepository . save (account);
}
```

This code might be correct from a business perspective, but if the tests don't clearly check the edge case, the mutant replacing `<0` with `<=0`, or reversing the logic, can persist. The problem, therefore, lies not only in the code itself, but also in the weakness of the tests in protecting this rule.

Corrected code and enhanced testing

The industry code can be slightly tightened:

```
audience void debitAccount ( Account account, BigDecimal amount)
{
    if (account == null ) {
        throw new BusinessException ( "Account must not be null"
);
    }
}
```

```

}

    if (amount == null || amount.signum ( ) <= 0 ) {

        throw new BusinessException ( "Debit amount must be
greater than zero" );
    }

    if ( account . getBalance ( ) . compareTo (amount) < 0 ) {

        throw new BusinessException ( "Insufficient balance for
transfer" );
    }

    account . setBalance ( account.getBalance ( ) . subtract
(amount));

    accountRepository . save (account);
}

```

But the main remediation must also involve a more precise unit test:

```

@ Test

void shouldThrowExceptionWhenBalanceIsInsufficient () {

    Account account = new Account ( "DB-TEST001" , new)
BigDecimal ( "5000.00" ), AccountType . CURRENT , 1L );

```

```

        BusinessException exception = assertThrows (
            BusinessException.class ,

            () -> accountService . debitAccount (account, new BigDecimal (
                "6000.00" ))

        );

        assertEquals ( "Insufficient balance for transfer" ,
            exception . getMessage ());
    }

    @ Test

    void shouldDebitAccountWhenBalanceIsSufficient () {

        Account account = new Account ( "DB-TEST002" , new
            BigDecimal ( "5000.00" ), AccountType . CURRENT , 1L );

        accountService . debitAccount (account, new BigDecimal (
            "1000.00" ));

        assertEquals ( new BigDecimal ( "4000.00" ), account .
            getBalance ());

        verify (accountRepository). save (account);
    }

```

```

@ Test

void shouldThrowExceptionWhenAmountIsZeroOrNegative () {

    Account account = new Account ( "DB-TEST003" , new
BigDecimal ( "5000.00" ), AccountType . CURRENT , 1L );

    assertThrows ( BusinessException . class ,

() -> accountService . debitAccount ( account, BigDecimal.ZERO )
);

    assertThrows ( BusinessException . class ,

() -> accountService . debitAccount (account, new BigDecimal (
"-10.00" ))) ;
}

```

Explanation of the correction

This remediation clearly demonstrates the complementary nature of security, robustness, and test quality. The service is made more defensive, but more importantly, the tests are enhanced to explicitly protect critical branches. Only under these conditions will PITest cease to consider certain mutants as survivors.

In a DevSecOps course, this step is essential because it teaches students that a project is not truly secure if it relies on sensitive business rules that tests cannot defend against simple logical alterations. Mutation testing then becomes a very concrete indicator of the maturity of the application's underlying infrastructure.

Verification commands

```

mvn clean test

```

The student must then compare the PITest report before and after correction, focusing particularly on the business services of the account and transfer modules. The goal is not necessarily to achieve a perfect score, but to reduce the number of surviving mutants on critical rules.

Impact on tools

PITest is the tool directly involved here. The mutation score should improve if the new tests effectively cover the sensitive branches and eliminate the previously surviving mutants.

SonarQube can also indirectly benefit from this improvement, particularly through coverage metrics and the overall picture of better-tested code. Dependency-Check is not impacted.

4.8 Expected outcome of Part 4

At the end of this fourth part, the student should no longer simply have read alerts or run tools. They should have concretely transformed DigiBank into a more robust version, in which the main weaknesses identified by the static analysis have been corrected in a justified, verifiable, and consistent manner with the project's multi-module architecture.

Specifically, he must be able to explain:

- Why secrets should no longer be displayed in plain text in the configuration
- Why stricter DTO validations improve security and business quality
- Why error messages should be helpful without being too revealing;
- Why critical transfer and debit rules must be defended at the service level
- Why a vulnerable dependency constitutes a system weakness even if the local code appears correct
- Why a good coverage score is not enough if tests allow mutants to survive on sensitive branches.

The corrected version of DigiBank thus becomes a much more credible basis for the rest of the course. It is better prepared for workshop 3 on DAST, cleaner for future work on Docker and GitHub Actions, and more aligned with the central logic of the course: detect early, fix



early, document clearly, and progressively industrialize security throughout the entire software lifecycle.

Part 5: Final validation, consolidation of results and preparation for the next stage of the process

Following the detailed identification of static vulnerabilities in DigiBank's source code and the reasoned remediation of the identified weaknesses, Workshop 2 must now conclude with a final validation phase, consolidating the acquired knowledge and methodically preparing for the next steps in the DevSecOps journey. This last part is essential, as security work cannot be considered complete simply because corrections have been made to the code or configuration. It is still necessary to verify that these corrections have effectively reduced the risk without degrading the system's functional behavior, that they have been coherently integrated into the project architecture, and that they leave behind a more robust foundation for future analyses. The Workshop 2 document clearly outlines this logic, demonstrating that DigiBank must evolve from a purely functional application to an instrumented, monitored, hardened system, ready to undergo more advanced controls later in the course.

5.1 Role of final validation in workshop 2

The final validation process confirms that the remediation implemented was not limited to a local or opportunistic fix, but that it effectively improved the overall security of the DigiBank project. From a pedagogical perspective, this step allows for a shift from a one-off fix approach to a technical mastery approach. Fixing a vulnerability is not simply a matter of modifying a few lines of code; it is demonstrating that the identified weakness has been understood, properly addressed, re-tested, and then integrated into a broader quality assurance process.

This phase is also important because a patch can introduce new side effects. Stricter input validation, for example, can break a previously tolerated business use case. An updated dependency can alter certain execution behaviors. A less verbose exception can disrupt existing tests if they relied on overly specific internal messages. Therefore, the end



of the workshop must necessarily include a cross-check of the build, tests, analysis reports, and overall project consistency.

5.2 What needs to be checked after remediation

Following the correction phase, several aspects must be explicitly rechecked. The first concerns the project's technical stability. DigiBank must still compile cleanly, its Maven modules must remain consistent, and existing tests must continue to run without unjustified regressions. This requirement stems directly from the logic of Lab 2, which emphasizes that a corrected version must remain functional while also being more secure.

The second dimension concerns the results of the SAST tools themselves. Alerts deemed critical or relevant in parts 3 and 4 must decrease, disappear, or be reclassified after remediation. This does not mean requiring a completely empty report, as some signals may stem from technical trade-offs, tool limitations, or false positives. However, priority vulnerabilities must no longer remain in their original form.

The third dimension concerns the quality of the justifications. In an academic DevSecOps framework, security is not simply the raw result of a scan. Students must be able to explain why a particular alert was corrected, why another was downplayed, and how the chosen remediation measures actually improve DigiBank's security posture. This ability to justify actions is an integral part of the objectives of Workshop 2.

5.3 Expected technical checks

Final validation must rely on a clear sequence of technical checks. The project must first be rebuilt cleanly from its Maven root. Then, the unit tests must be rerun to ensure that the remediation changes haven't broken the core business rules. Lab 2 already reminds us that Maven is the natural entry point for these checks and that the clean, test, and verify cycle structures the reproducibility of the analysis.

Similarly, the security tools already integrated into the workshop must be relaunched after the fixes. SonarQube should be used to monitor the evolution of security vulnerabilities, hotspots, and certain structural weaknesses. OWASP Dependency-Check should verify that problematic dependencies have been updated, replaced, or documented. PITest should be rerun if the fixes affect sensitive business areas, to ensure that the tests

remain capable of detecting significant logical alterations. Lab 2 specifically presents these tools as complementary and intended for use within a coherent static control chain.

A typical final verification sequence can therefore be presented as follows:

```
mvn clean

mvn test

mvn clean verify

mvn org.owasp:dependency-check-maven:check

mvn org.pitest:pitest-maven:mutationCoverage

mvn clean verify sonar:sonar -Dsonar.login=YOUR_SONAR_TOKEN
```

These commands are not merely an execution routine; they embody the DevSecOps logic of repeatability, automation, and continuous validation that runs throughout the entire shop floor.

5.4 Acceptance criteria for a corrected version of DigiBank

A corrected version of DigiBank can be considered acceptable at the end of Workshop 2 if several criteria are met simultaneously. The first criterion is the absence of major functional regressions. A secure application that has become unstable or incorrect from a business perspective would not meet the learning objectives of the exercise. Security should enhance the solution, not invalidate its functionality.

The second criterion is the effective reduction of prioritized vulnerabilities. If the critical alerts identified in the detailed analysis section are still present without solid justification, remediation cannot be considered complete. TP2 also emphasizes that the workshop's objective is to deliver a more robust, better-controlled, and technically safer version than the one produced in Workshop 1.

The third criterion is the architectural consistency of the fixes. Remediation must respect DigiBank's modular organization, integrate cleanly into controllers, services, DTOs, validations, and configuration files, and not create new security debt under the guise of



addressing an existing vulnerability. A quality fix is therefore one that is effective, clean, and maintainable.

5.5 Capitalization and traceability of corrections

The end of workshop 2 must also include a knowledge-sharing component. In a DevSecOps approach, it's not enough to simply fix things; you need to keep a clear record of what has been fixed, what remains to be observed, and the reasons behind the decisions made. Workshop 2 already highlights the importance of Git, Maven, and GitHub Actions in ensuring the reproducibility and traceability of changes applied to the project.

This knowledge sharing can take several complementary forms. First, Git commits must reflect understandable remediation intentions, not simply opaque changes. Second, the workshop report or associated technical note must explicitly state the identified vulnerabilities, their criticality level, the actions taken, and the results obtained after correction. Finally, when an alert is retained without immediate correction, this must be documented with a reasoned explanation, for example, due to a false positive, a transitive dependency that is difficult to replace at this stage, or a deliberate educational decision.

This traceability has a strong educational value. It teaches the student that professional security depends as much on the quality of reasoning and documentation as on the writing of the correction itself.